Invoice Studio API

Route reference, authentication and test guide for the Azure deployment. Generated from the app's own route table: the access level shown is the one the server enforces.

1. Base URL and access levels

`https://<app>.<region>.azurecontainerapps.io` (the `appUrl` output of the deploy), or the custom host name. HTTPS only: plain HTTP is redirected. Swagger, ReDoc and `/openapi.json` are switched off in the cloud build.

Level	Who	How
No token	anyone	`/health` and the page itself; no data
Public	anyone	`GET /api/public-config` only: sign-in settings, no secret, no data
Member	users with the `member` or `admin` app role	Entra ID bearer token
Admin	users with the `admin` app role	Entra ID bearer token; 60 admin writes per minute per user

The role is read from the token on every request. The app's user table records sign-ins and grants nothing: roles are given by assigning people (or their group) to the app role in Entra ID.

2. Getting a token

The API accepts only Entra ID v2 access tokens that the Invoice Studio app registration itself requested: RS256, issuer `https://login.microsoftonline.com/<tenant>/v2.0`, audience the app's client id, scope `access_as_user`, `azp` the app's client id, not expired. A token for any other client (Azure CLI, Graph Explorer, Postman's own client) is refused with 401 by design, so the token comes from the app's sign-in in the browser:

1. Open the app and **Sign in with Microsoft**.

2. Open the browser developer tools (F12) -> **Network**, and click any request to `/api/` (for example `state`).

3. Under **Request Headers**, copy the value of `Authorization` after `Bearer ` (it starts with `eyJ`).

4. Paste it into the Postman environment's `token` (and a member's token into `memberToken`).

A token lasts about an hour. Never paste a token into a chat, a ticket or a commit: it is the user's access.

3. Headers

Header	When	Value
`Authorization`	every Member and Admin call	`Bearer <token>`
`X-Studio-Request`	every POST, PUT, PATCH and DELETE	`1`. A write without it is refused with 403 (request forgery protection).
`Origin`	sent by browsers	must be the app's own address. Postman and curl send none, which is accepted.
`Content-Type`	JSON bodies	`application/json`; uploads are `multipart/form-data`

Every response carries `X-Request-Id`; quote it when reporting a problem, it is in the app log. Responses also carry HSTS (one year), `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`, `Cache-Control: no-store` and a Content-Security-Policy with `frame-ancestors 'none'`.

4. Answers and errors

Status	Meaning
200	Done. JSON, or a file for downloads.
400	The request is wrong; `detail` says why in plain words.
401	No token, an expired token, or a token not issued to this app.

Status	Meaning
403	Signed in but not allowed: member on an admin route, write without `X-Studio-Request`, foreign `Origin`, AI turned off, or an AI provider this deployment does not allow.
404	No such route or record. An unknown `/api/` path is always 404.
409	The state changed: plan not confirmed or not yours, stale revision, invoice still reading or already exported.
413	Upload over 12 MB.
422	The body does not match the schema (unknown or missing field, wrong type).
429	Rate limit: admin writes, too many plans, or the reading queue is full. Wait and retry.
500, 503	Server side. The body has the request id, never a trace. `/api/health/dependencies` answers 503 when the database or storage cannot be reached.

Limits: 12 MB per file, 500 files per plan, a confirmed plan stays valid 10 minutes after its last use.

5. Testing a deployment

Postman. Import both files from `docs/api/`, choose the *Invoice Studio* environment, set `baseUrl` and `token` (and `memberToken` if you have a member account), then run the folders in order:

Folder	What it proves	Changes data?
1. Smoke test	The app, sign-in, database and storage answer; security headers are present	No
2. Security checks	No token, forged token, missing request header, foreign origin, unknown route, schema pages, member on admin route, non-Azure AI provider: each refused	No (each write is refused)
3. Invoice flow	Plan, confirm, upload the synthetic sample, read it locally (no AI), audit trail, then delete it	Creates and deletes one test invoice (needs an admin token)
4. API reference	Every route with an example; for reading, not for running as a whole	Yes, if run

For the upload in folder 3, Postman needs `samples/invoice.pdf` from this repository: set Postman's working directory to the checkout, or select the file on the request.

From Azure Cloud Shell, without Postman (Cloud Shell has Node.js):

```
cd ~/Invoice_studio_Al
read -rs TOKEN && export TOKEN          # paste the token, Enter (stays out of the history)
npx --yes newman@6 run docs/api/invoice-studio.postman_collection.json \
  -e docs/api/invoice-studio.postman_environment.json --working-dir . \
  --env-var baseUrl=https://<app>.<region>.azurecontainerapps.io --env-var token="$TOKEN" \
  --folder "1. Smoke test" --folder "2. Security checks" --folder "3. Invoice flow"
```

Every check prints a tick or a cross, and newman exits non-zero if any failed. For the full attack pack (70+ routes without a token, forged tokens, hostile files, floods) see `docs/ATTACK_PACK.md`.

Routes

Health and sign-in

Public health check, sign-in settings and the signed-in session.

Method	Path	Access	What it does
GET	`/health`	**No token**	**Health check.** Answers {"status":"ok","version":...} once the app is up. No token. Used by the platform probe.
GET	`/api/public-config`	**Public**	**Sign-in settings.** Tenant id, client id, scope and authority the browser needs to sign in. Public by design; holds no secret and no data.
GET	`/api/session`	**Member**	**Who am I.** The signed-in user as the API sees them: id, name, email and role (admin or member), read from the token on this request.
GET	`/api/state`	**Member**	**Workspace state.** Everything the main screen shows: settings, connections (never keys), references summary, recent invoices and exports.
GET	`/api/queue`	**Member**	**Processing queue.** Invoices queued and being read on this instance, and how many it admits.
GET	`/api/health/dependencies`	**Member**	**Readiness of database and storage.** Reads the database and lists the evidence container once each, through the private endpoints when they are locked down. 200 with ok for each, or 503 with the failed one; never an address or message.

Processing plan and upload

Every upload runs under a processing plan the same person previewed and confirmed: preflight, confirm, then upload each file named in the plan.

Method	Path	Access	What it does
POST	`/api/preflight`	**Member**	**Preview a processing plan.** Send the options and the name and size of every file. Returns a plan token, a summary and warnings. Up to 500 files of up to 12 MB each.
POST	`/api/preflight/confirm`	**Member**	**Confirm the plan.** Confirms the previewed plan. Only the account that made the plan can confirm it.
POST	`/api/preflight/renew`	**Member**	**Keep a plan alive.** Extends a confirmed plan by 10 minutes while a long batch is still uploading.
POST	`/api/invoices`	**Member**	**Upload one invoice.** Multipart form: file, options (the same JSON as the plan) and preflight_token. The file must be named in the confirmed plan with the same size. PDF, PNG, JPEG, TIFF, DOCX, XLSX, CSV or text; 12 MB at most. Returns the invoice record; reading continues in the background.

```
POST /api/preflight
{
 "options": {
  "engine": "auto",
  "prefer_native_text": true,
  "ai_fallback": false,
  "ai_cross_check": false,
  "provider": "azure",
  "model": "",
  "language": "en"
 },
 "files": [
  {
   "name": "invoice.pdf",
   "size": 2140
  }
 ]
}

POST /api/preflight/confirm
{
 "token": "{{planToken}}"
}
```

```
POST /api/preflight/renew
{
 "token": "{{planToken}}"
}

POST /api/invoices
multipart form:
  file = @samples/invoice.pdf
  options = {
     "engine": "auto",
     "prefer_native_text": true,
     "ai_fallback": false,
     "ai_cross_check": false,
     "provider": "azure",
     "model": "",
     "language": "en"
    }
  preflight_token = {{planToken}}
```

Invoices

Read one invoice, its source document, audit trail and target check.

Method	Path	Access	What it does
GET	`/api/jobs/{jid}`	**Member**	**One invoice.** Status (queued, processing, needs_review, ready, exported, error), extracted fields, lines, checks and the reader trace.
GET	`/api/jobs/{jid}/document`	**Member**	**The uploaded document.** The original file, served with its own content type, nosniff and a sandboxing CSP.
GET	`/api/jobs/{jid}/audit`	**Member**	**Audit trail of one invoice.** Every step recorded for this invoice: upload hash, reads, edits, approvals, exports.
GET	`/api/jobs/{jid}/target-check`	**Member**	**Target check.** Read-only: the invoice's target-sheet cells by status, with evidence and row checks.
GET	`/api/target-check/accuracy`	**Member**	**Target-check accuracy.** Confirm-time accuracy against owner truth, per field. Optional ?supplier=.

```
GET /api/target-check/accuracy
query: supplier=
```

Review and export

Review edits, retry, approve and export one invoice.

Method	Path	Access	What it does
POST	`/api/jobs/{jid}/review`	**Member**	**Save review edits.** Saves edited fields and lines for the given revision; confirm:true approves. A stale revision is refused with 409.
POST	`/api/jobs/{jid}/retry`	**Member**	**Read again.** Reads the invoice again under a new confirmed plan.
POST	`/api/jobs/{jid}/export`	**Member**	**Export one invoice.** Creates the Excel export for an invoice at the given revision: the three template sheets with what was found, blanks blank. Open validation and review holds do not block it (owner decision D11); the receipt records them as open_holds. 409 only if it is still processing, not read, changed or a duplicate; an invoice already exported returns its existing export.
POST	`/api/jobs/{jid}/extraction-draft`	**Member**	**Unvalidated extraction workbook.** Downloads what was read, before validation, as Excel. Needs acknowledge_unvalidated.
POST	`/api/jobs/{jid}/draft`	**Member**	**Manual draft workbook.** Builds a target workbook from hand-entered Header, Tax_Breakdown and Details. Never approves.

```
POST /api/jobs/{jid}/review
{
 "invoice": {
  "number": "INV-1"
 },
 "revision": 1,
 "confirm": false
}
```

```
POST /api/jobs/{jid}/retry
{
 "options": {
  "engine": "auto",
  "prefer_native_text": true,
  "ai_fallback": false,
  "ai_cross_check": false,
  "provider": "azure",
  "model": "",
  "language": "en"
 },
 "preflight_token": "{{planToken}}"
}

POST /api/jobs/{jid}/export
{
 "revision": 1
}

POST /api/jobs/{jid}/extraction-draft
{
 "revision": 1,
 "acknowledge_unvalidated": true
}

POST /api/jobs/{jid}/draft
{
 "acknowledge_unvalidated": true,
 "Header": {},
 "Tax_Breakdown": {},
 "Details": []
}
```

Exports and downloads

Download exports and batch workbooks; long downloads run in the background.

Method	Path	Access	What it does
GET	`/api/exports/{eid}`	**Member**	**Download an export.** The export workbook.
GET	`/api/exports/{eid}/evidence`	**Member**	**Evidence of an export.** The evidence record behind an export: sources, hashes, approvals.
POST	`/api/exports/batch`	**Member**	**Export a batch.** One workbook for several invoices; each item names the invoice and its revision. Holds do not block (owner decision D11): each invoice is written with what was found, blanks blank, and its receipt records the open holds for the export team to validate by hand. All or nothing on integrity: an invoice still processing or not read, changed, already exported or duplicated refuses the batch with 409. The workbook holds only the Header, Tax_Breakdown and Details sheets, Transaction Number 1..n (no Checks sheet).
POST	`/api/exports/batch/check`	**Member**	**Check a batch before export.** Read-only. Splits the selected invoices into ready (in transaction order, each with attention: the holds still open on it, information only since D11) and blocked, each blocked one with its reasons as code and message (Processing, Not Read, Changed, Exported, Duplicate). Send the ready ones to /api/exports/batch.
GET	`/api/batches/{bid}`	**Member**	**Download a batch export.** The batch workbook.
POST	`/api/exports/extraction-batch`	**Member**	**Unvalidated batch workbook.** What was read for several invoices, before validation.
POST	`/api/exports/target-draft`	**Member**	**Target draft for unapproved invoices.** The draft of everything: the target workbook for all selected invoices, held ones included, with the Checks sheet. Marked DRAFT in the file name and on every sheet (red tab and header row, a note on A1, the printed header); sheet names and headers stay the template's. Never an export.
POST	`/api/background/{kind}`	**Member**	**Start a long download.** kind is target-draft, extraction-batch, export-check, export-batch, fine-rules-target or fine-rules-review; the body is that download's body. Returns a task id to poll.
GET	`/api/background/tasks/{tid}`	**Member**	**Poll a long download.** Running, or the finished file. Only the account that started it can read it.

```
POST /api/exports/batch
{
 "jobs": [
  {
   "id": "{{jobId}}",
   "revision": 1
  }
 ]
}

POST /api/exports/batch/check
{
 "jobs": [
  {
   "id": "{{jobId}}",
   "revision": 1
  }
 ]
}

POST /api/exports/extraction-batch
{
 "jobs": [
  {
   "id": "{{jobId}}",
   "revision": 1
  }
 ],
 "acknowledge_unvalidated": true
}

POST /api/exports/target-draft
{
 "jobs": [
  {
   "id": "{{jobId}}",
   "revision": 1
  }
 ]
}

POST /api/background/{kind}
{
 "jobs": [
  {
   "id": "{{jobId}}",
   "revision": 1
  }
 ]
}
```

Fine rules and target export

Mapping-table driven checks, review and target workbooks.

Method	Path	Access	What it does
GET	`/api/fine-rules/config`	**Member**	**Fine-rules configuration.** The active mapping-table configuration.
POST	`/api/fine-rules/config`	**Admin**	**Save fine-rules configuration.** Replaces the configuration (JSON object). Audited by field names and sizes, never values.
POST	`/api/fine-rules/run`	**Member**	**Run fine rules.** Runs the rules over invoices.
POST	`/api/fine-rules/review.xlsx`	**Member**	**Fine-rules review workbook.** The rule findings as Excel.
POST	`/api/fine-rules/target.xlsx`	**Member**	**Fine-rules target workbook.** The target workbook after the rules.
GET	`/api/fine-rules/feedback`	**Member**	**Rule feedback.** Corrections sent by reviewers.
POST	`/api/fine-rules/feedback`	**Member**	**Send rule feedback.** A reviewer's correction to a rule outcome, with evidence.
POST	`/api/fine-rules/feedback/{fid}/decision`	**Admin**	**Decide on feedback.** Admin accepts or rejects a correction.
GET	`/api/rules-assistant`	**Member**	**Rules assistant state.** Whether the AI rules assistant is on and, if not, why.

Method	Path	Access	What it does
POST	`/api/rules-assistant/test`	**Admin**	**Test the rules assistant connection.** Admin check of the Azure AI Foundry connection; the assistant stays off until it passes.
POST	`/api/rules-assistant/search`	**Member**	**Rules assistant: search the item master.** Ranked item master rows for a code or description. No AI call.
POST	`/api/rules-assistant/explain`	**Member**	**Rules assistant: explain a match.** Why an invoice line matched an item master row or not, from the engine's facts.
POST	`/api/rules-assistant/propose`	**Member**	**Rules assistant: propose rules.** Drafts rule rows for a line and logs them as Proposed rule feedback; an admin decides.
GET	`/api/target-export/config`	**Member**	**Target export settings.** Which UPC column the target export fills.
POST	`/api/target-export/config`	**Admin**	**Save target export settings.** Admin sets the UPC column.

```
POST /api/fine-rules/config
{}

POST /api/fine-rules/run
{
 "job_ids": [
  "{{jobId}}"
 ]
}

POST /api/fine-rules/review.xlsx
{
 "job_ids": [
  "{{jobId}}"
 ]
}

POST /api/fine-rules/target.xlsx
{
 "job_ids": [
  "{{jobId}}"
 ]
}

POST /api/fine-rules/feedback
{
 "invoice_line": "1",
 "original": "",
 "correction": "x",
 "reason": "",
 "evidence": "page 1"
}

POST /api/fine-rules/feedback/{fid}/decision
{
 "decision": "accepted",
 "approver": "",
 "version": ""
}

POST /api/rules-assistant/search
{
 "query": "rose cream"
}

POST /api/rules-assistant/explain
{
 "line": {
  "description": "Rose hand cream 50ml",
  "gtin": "",
  "sku": ""
 }
}
```

```
POST /api/rules-assistant/propose
{
 "line": {
  "description": "Rose hand cream 50ml",
  "gtin": "",
  "sku": ""
 },
 "note": ""
}

POST /api/target-export/config
{
 "upc": "empty"
}
```

Product and PO lookup

Search the imported item master and PO/GRN records.

Method	Path	Access	What it does
GET	`/api/reference-lookup/summary`	**Member**	**Lookup summary.** Counts of the imported item master and PO/GRN records.
GET	`/api/reference-lookup/search`	**Member**	**Search items or PO/GRN.** kind=item or po, q of 2 to 100 characters; paged with cursor and limit (1-50).
GET	`/api/reference-lookup/products`	**Member**	**Product candidates.** Ranked item-master candidates for an invoice line.

```
GET /api/reference-lookup/search
query: kind=item&q=widget&limit=25

GET /api/reference-lookup/products
query: q=widget&limit=10
```

Learned layouts

Supplier layouts learned from reviewed invoices.

Method	Path	Access	What it does
GET	`/api/learned`	**Member**	**Learned layouts.** Layouts learned so far.
POST	`/api/learned/jobs/{jid}`	**Member**	**Learn from an invoice.** Learns the supplier layout from a reviewed invoice.
DELETE	`/api/learned/{key}`	**Admin**	**Forget a layout.** Admin removes a layout.

Sample files

Synthetic sample files for training and tests.

Method	Path	Access	What it does
GET	`/api/samples/reference.xlsx`	**Member**	**Sample reference workbook.** Synthetic reference data.
GET	`/api/samples/invoice.pdf`	**Member**	**Sample invoice.** A synthetic invoice.
GET	`/api/samples/template.yml`	**Member**	**Sample supplier template.** An invoice2data template.

Item master and mapping tables

Admin uploads of the owner's master data, with validation, activation and rollback.

Method	Path	Access	What it does
GET	`/api/item-master`	**Admin**	**Item master versions.** Uploaded and active item-master versions.
POST	`/api/item-master/validate`	**Admin**	**Validate an item master.** Multipart file upload; validates without activating.
POST	`/api/item-master/uploads/{version}/activate`	**Admin**	**Activate an item master.** Makes a validated upload the active version.
POST	`/api/item-master/versions/{version}/rollback`	**Admin**	**Roll back the item master.** Returns to an earlier version; ?expected_signature= guards against a concurrent change.
DELETE	`/api/item-master/uploads/{version}`	**Admin**	**Discard an upload.** Removes a validated upload that was never activated.

Method	Path	Access	What it does
POST	`/api/item-master/refresh`	**Admin**	**Update item master.** Re-matches every open (not exported) invoice against the live item master in the background; activate and rollback start this too.
GET	`/api/item-master/refresh`	**Admin**	**Item master refresh result.** The latest re-match: status, counts and, per changed invoice, the lines newly matched, lost or changed.
GET	`/api/mapping-tables`	**Admin**	**Mapping-table versions.** Uploaded and active mapping-table versions.
POST	`/api/mapping-tables/validate`	**Admin**	**Validate mapping tables.** Multipart file upload; validates without activating.
POST	`/api/mapping-tables/uploads/{upload_id}/activate`	**Admin**	**Activate mapping tables.** Makes a validated upload active.
POST	`/api/mapping-tables/versions/{version_id}/rollback`	**Admin**	**Roll back mapping tables.** Returns to an earlier version.
GET	`/api/mapping-tables/active/workbook`	**Admin**	**Download active mapping tables.** The active workbook.

```
POST /api/item-master/validate
multipart form:
  file = @item-master.xlsx

POST /api/item-master/versions/{version}/rollback
query: expected_signature=

POST /api/mapping-tables/validate
multipart form:
  file = @mapping-tables.xlsx
```

Reference data and policy

Admin setup: references, supplier templates, tolerances.

Method	Path	Access	What it does
POST	`/api/references`	**Admin**	**Import references.** Multipart file upload of the reference workbook.
POST	`/api/references/demo`	**Admin**	**Load demo references.** Synthetic references.
POST	`/api/demo`	**Admin**	**Run the demo invoice.** Reads the synthetic invoice under a confirmed plan.
POST	`/api/templates`	**Admin**	**Add a supplier template.** Multipart upload of an invoice2data YAML template.
POST	`/api/policy`	**Admin**	**Save tolerances.** Price and total tolerances and currency decimals. Marks unexported invoices for review again.

```
POST /api/references
multipart form:
  file = @reference.xlsx

POST /api/demo
{
 "preflight_token": "{{planToken}}"
}

POST /api/templates
multipart form:
  file = @template.yml

POST /api/policy
{
 "price_tolerance": "0",
 "total_tolerance": "0.01",
 "currency_decimals": {
  "AED": 2
 }
}
```

AI connection and settings

Admin control of AI. The Azure deployment allows Azure AI Foundry only (INV_STUDIO_AI_PROVIDERS=azure); other providers answer 403.

Method	Path	Access	What it does
POST	`/api/settings`	**Admin**	**AI settings.** Default provider and model, and the AI on/off switch (ai_fallback). With it off the server refuses every AI read.
GET	`/api/ai-rules`	**Member**	**Rules the AI added.** The rules the AI added, each with its source, reason, model, file and the lines it has filled. There is no approval step.
DELETE	`/api/ai-rules/{rule_id}`	**Admin**	**Remove a rule the AI added.** Removes one rule the AI added, so it fills nothing more. Audited.
POST	`/api/connections/azure`	**Admin**	**Connect Azure AI Foundry.** Endpoint (an Azure resource host or Foundry project), agent or deployment name, and an api-key or managed identity. Makes a real call first: only a completed answer saves it, stored encrypted; a failed test saves nothing and keeps the previous connection.
DELETE	`/api/connections/azure`	**Admin**	**Disconnect Azure AI Foundry.** Removes the saved connection.
POST	`/api/connections/{provider}`	**Admin**	**Connect OpenAI or Anthropic.** Refused with 403 on the Azure deployment.
DELETE	`/api/connections/{provider}`	**Admin**	**Disconnect OpenAI or Anthropic.** Removes a saved key.
GET	`/api/connections/{provider}/models`	**Admin**	**List models.** Models of a connected provider. 403 for a provider the deployment does not allow.
POST	`/api/subscriptions/claude/import`	**Admin**	**Import a Claude subscription.** Local installs only in practice; 403 on the Azure deployment.
DELETE	`/api/subscriptions/claude`	**Admin**	**Remove the Claude subscription.** Removes it.
POST	`/api/chatgpt/start`	**Admin**	**ChatGPT sign-in (local only).** Refused in the cloud build.
POST	`/api/chatgpt/select`	**Admin**	**Select a ChatGPT account.** 403 on the Azure deployment.
POST	`/api/chatgpt/export`	**Admin**	**Export a ChatGPT credential bundle.** Refused in the cloud build.
POST	`/api/chatgpt/import`	**Admin**	**Import a ChatGPT credential bundle.** 403 on the Azure deployment.
DELETE	`/api/chatgpt/{aid}`	**Admin**	**Remove a ChatGPT account.** Removes it.

```
POST /api/settings
{
 "provider": "azure",
 "model": "",
 "ai_fallback": false
}

POST /api/connections/azure
{
 "endpoint": "https://<resource>.openai.azure.com",
 "name": "<deployment or agent>",
 "auth": "key",
 "api_key": "<key>"
}

POST /api/connections/{provider}
{
 "api_key": "<key>"
}

POST /api/subscriptions/claude/import
{
 "setup_token": "<token>"
}

POST /api/chatgpt/start
{
 "account_id": null
}

POST /api/chatgpt/select
{
 "id": "<account>"
}
```

```
POST /api/chatgpt/export
{
 "id": "<account>"
}

POST /api/chatgpt/import
{
 "bundle": {},
 "passphrase": "<passphrase>"
}
```

Administration

Delete invoices, costs, users.

Method	Path	Access	What it does
POST	`/api/jobs/delete`	**Admin**	**Delete invoices.** Permanently deletes invoices and their files. Each item names the invoice and its current revision; needs confirm_permanent: true (a boolean).
GET	`/api/costs`	**Admin**	**AI spend.** Daily and month-to-date AI spend. ?days=30.
GET	`/api/users`	**Admin**	**Users.** Everyone who has signed in, their role and last sign-in. Read-only: roles are granted in Entra ID.

```
POST /api/jobs/delete
{
 "jobs": [
  {
   "id": "{{jobId}}",
   "revision": 1
  }
 ],
 "confirm_permanent": true
}

GET /api/costs
query: days=30
```

Totals: 88 routes: 1 public, 47 member, 39 admin, 1 without a token.